Joint Activation and Structured Sparsity for PlantVillage on Akida 1

Akida 1 Model Zoo

August 2026

Abstract

Akida 1 hardware accelerates inference by exploiting *activation* (event) sparsity, not weight sparsity. A companion study on Visual Wake Words (VWW) found that combining activation-sparsity regularization with structured (channel) pruning is *complementary*: the combined accuracy cost is less than the sum of each mechanism's individual cost. This report repeats that investigation on PlantVillage (38-class plant-disease classification, AkidaNet $\alpha = 0.5$) and finds the *opposite* at aggressive settings: the joint configuration costs 19.58 accuracy points, an order of magnitude more than either mechanism alone ($\sim$1.4 points each). Tracing the cause shows this is primarily a fine-tuning-budget problem rather than fundamentally incompatible mechanisms — quadrupling the post-prune fine-tuning recovers most (not all) of the gap — and that a *lighter* joint configuration avoids the problem almost entirely, reaching 97.81% accuracy (a 1.92-point drop) with 75.9% activation sparsity and a 21.0% parameter reduction using the same modest fine-tuning budget that left the aggressive configuration badly undertrained. The practical conclusion mirrors VWW's (a lighter joint configuration is the best operating point) but for a different reason: here, aggressive joint settings are a genuine trap rather than merely a less-efficient use of accuracy budget.

Contents

1 Introduction

This report repeats, on a second model and dataset, the joint activation-sparsity and structured-sparsity investigation first conducted on Visual Wake Words (VWW). It asks the same two questions as the original study:

(i) Does combining activation-sparsity regularization with structured channel pruning cost *more* accuracy than the sum of what each costs on its own, or can the two be pushed together more cheaply?

(ii) Can a better-optimized operating point be found that keeps most of the sparsity/size benefit while shrinking the accuracy drop?

On VWW, the answer to (i) was that the two mechanisms are complementary: the joint configuration cost *less* than the sum of its parts. This report finds that answer does **not** generalize as-is: on PlantVillage, an aggressive joint configuration costs dramatically *more* than the sum of its parts. Diagnosing why (Section 7) shows this is substantially — though not entirely — a fine-tuning-budget artifact, and that a lighter joint configuration avoids it, giving the same practical recommendation as VWW (favor a lighter joint configuration) by a different route.

All work in this report is on the branch `akida1-sparsify-plantvillage-joint`. This follows up on two prior efforts: the single-axis activation-sparsity experiment on branch `akida1-sparsify-plantvillage` (`SPARSITY_EXPERIMENT.md`), and the joint VWW study on branch `akida1-sparsify-vww-joint`.

2 Background

2.1 Akida 1 and activation sparsity

Akida 1 is an event-based neural processor that skips computation on zero-valued activations, so models with sparser intermediate activations run faster and at lower power on the physical device. Structured (channel) sparsity is a different property: it describes how much of the model's parameter count has been physically removed. Akida 1's execution model does not exploit weight sparsity for speed, so reducing parameter count is a memory-footprint argument, not a latency one.

2.2 PlantVillage

PlantVillage is a 38-class image classification task: given a leaf image, predict which of 38 (crop, disease) categories it belongs to (including healthy classes). It is a substantially larger and more

fine-grained task than VWW's binary person/non-person classification, both in output cardinality and in input resolution.

3 Dataset

The dataset is the *Plant leaf diseases dataset without augmentation*, loaded via `tensorflow_datasets` from a mirror hosted at `data.brainchip.com` (`plant_village_data.py`). It contains approximately 54,000 leaf images across 38 classes.

Table 1: Dataset summary, as loaded by `plant_village_data.get_data`.

Property	Value
Classes	38 (plant/disease combinations)
Input resolution	$224 \times 224 \times 3$ (RGB, `uint8`)
Train / validation / test split	80% / 10% / 10%
Total images	$\approx$54,000
Train-time augmentation	random horizontal flip, brightness ($\pm$0.1), contrast (0.9–1.1)
Validation/test-time augmentation	none (only resize)

Unlike VWW, PlantVillage has a dedicated held-out *test* split, separate from validation; all accuracy figures in this report are measured on that test split, matching `plant_village_eval.py`'s own convention.

4 Model architecture

The model is AkidaNet at width multiplier $\alpha = 0.5$ (twice VWW's $\alpha = 0.25$), 224×224 input (VWW: 96×96), initialized from ImageNet-pretrained weights and adapted for PlantVillage with a deeper classification head than VWW's: the backbone's final feature layer feeds a 512-unit dense block (with its own BatchNormalization and ReLU) before a dropout layer and the 38-class logits layer. Like VWW, the backbone is a strict feed-forward chain with no residual/skip connections.

Table 2: AkidaNet-PlantVillage ($\alpha = 0.5$) architecture. Every `conv_*`/`separable_*` row is immediately followed by its own BatchNormalization and ReLU (omitted for brevity; parameter counts include them).

Layer	Type	Output shape	Params (layer + BN)
`input`	Input	$224 \times 224 \times 3$	0
`rescaling`	Rescaling ($\div 255$)	$224 \times 224 \times 3$	0
`conv_0`	Conv2D, stride 2	$112 \times 112 \times 16$	496
`conv_1`	Conv2D	$112 \times 112 \times 32$	4,736
`conv_2`	Conv2D, stride 2	$56 \times 56 \times 64$	18,688
`conv_3`	Conv2D	$56 \times 56 \times 64$	37,120
`separable_4`	SeparableConv2D, stride 2	$28 \times 28 \times 128$	9,280
`separable_5`	SeparableConv2D	$28 \times 28 \times 128$	18,048
`separable_6`	SeparableConv2D, stride 2	$14 \times 14 \times 256$	34,944
`separable_7`	SeparableConv2D	$14 \times 14 \times 256$	68,864
`separable_8`	SeparableConv2D	$14 \times 14 \times 256$	68,864
`separable_9`	SeparableConv2D	$14 \times 14 \times 256$	68,864
`separable_10`	SeparableConv2D	$14 \times 14 \times 256$	68,864
`separable_11`	SeparableConv2D	$14 \times 14 \times 256$	68,864
`separable_12`	SeparableConv2D, stride 2	$7 \times 7 \times 512$	135,424
`separable_13`	SeparableConv2D + global-avg-pool	512	268,800
`fc_1`	Dense + BN + ReLU	512	264,704
`dropout_1`	Dropout	512	0
`predictions`	Dense (logits, no activation)	38	19,494
		Total	**1,156,054**
		Trainable / non-trainable	1,149,046 / 7,008

Note that `fc_1`, unlike VWW's direct backbone-to-logits connection, sits between the backbone and the classifier head – one of the reasons `separable_13` is excluded from the prunable set (Section 5.2).

5 Methodology

The two sparsity mechanisms, and the rationale for a float-only pipeline, are identical to the VWW study; this section summarizes them and highlights the one PlantVillage-specific parameter (the established best activation-regularizer strength).

5.1 Activation sparsity: normalized Hoyer-Square regularization

An activity regularizer is attached to every ReLU layer's output x:

$$\mathcal{L}_{\text{Hoyer-norm}}(x) = \frac{\lambda}{N} \cdot \frac{\left(\sum_i |x_i|\right)^2}{\sum_i x_i^2 + \varepsilon}, \tag{1}$$

where N is the tensor's element count (the normalization that bounds the penalty to $(0, 1]$ regardless of layer width). The prior single-axis experiment on PlantVillage established $\lambda = 36$ as the best point on this axis alone: 78.2% activation sparsity (measured on the quantized, Akida-converted

model) for a ∼1.5-point accuracy cost. This λ is an order of magnitude larger than VWW's own best value ($\lambda = 3.6$) because PlantVillage's per-layer activation-tensor sizes are far more imbalanced (ranging up to ∼792× between the smallest and largest ReLU layer, versus VWW's ∼186×), so the same normalized penalty scale requires a proportionally larger λ to reach comparable pressure.

5.2 Structured sparsity: Network Slimming

Network Slimming [1] is applied identically to the VWW study: an L_1 penalty ($\mu = 10^{-5}$) on each targeted BatchNormalization layer's γ, attached via the zero-argument-callable form of `model.add_loss`; after training, channels are ranked by $|\gamma|$ and the lowest-ranked fraction (the prune target) is dropped per layer, then the model is physically rebuilt with sliced weights, propagating the channel-count change to the next weighted layer. The implementation (`plant_village_structured_sparsity.py`) is a direct, unmodified port of VWW's – it walks the model graph generically rather than hardcoding layer names or channel counts, so it applies to this different architecture without changes.

Pruning scope. `separable_4` through `separable_12` (9 of the network's 10 separable-convolution blocks) are eligible for pruning; the stem and `separable_13` are excluded, since `separable_13` feeds `fc_1` directly and pruning it would require resizing that Dense layer's input.

5.3 Float-only evaluation

As with VWW, this study trains and evaluates float models only – no `cnn2snn quantize`/QAT/`convert` anywhere in the pipeline – for the same two reasons: it sidesteps the software-simulator-vs-hardware sparsity discrepancy documented in the single-axis experiments, and it keeps channel pruning decoupled from quantization concerns. Activation sparsity is measured as the mean fraction of exactly-zero outputs across all ReLU layers on 1,000 held-out samples, and **is not directly comparable** to the prior experiment's Akida-measured figures for the same reason as VWW (e.g. $\lambda = 36$ reads 83.1% here on the float model versus 78.2% on the quantized/converted model).

5.4 Training pipeline

Each configuration is defined by (λ, p): the activation-regularizer strength and structured prune target.

Phase 1 (float training, 10 epochs). Matches `plant_village_train.sh`'s existing float-training schedule (Adam, learning rate 10^{-3} with exponential decay to $10^{-3} \times 0.01$ over the run) – this model already reaches 99%+ accuracy in 10 epochs, so no epoch-budget increase was needed before regularization was introduced, unlike VWW.

Pruning (if $p > 0$). Channels pruned per Section 5.2.

Phase 2 (post-prune fine-tuning, learning rate 10^{-4}). 5 epochs by default (proportionally similar to VWW's 40:15 Phase-1:Phase-2 ratio); Section 7 shows this default was insufficient at aggressive settings, and a 20-epoch variant was also tested.

Data pipeline. PlantVillage's `tf.data`-based loading pipeline (`plant_village_data.py`, via `tensorflow_datasets`, with `AUTOTUNE` prefetching already built in) did not exhibit VWW's single-threaded-generator bottleneck – no analogous `workers=`/`use_multiprocessing` fix was needed. A

full training epoch on the ∼43,000-image training split at 224×224 took approximately 40–45 seconds on GPU 1 of the shared training server.

6 Experimental design

Three rounds of configurations were run, sharing one summary CSV.

Pilot (2×2 grid). All four combinations of activation regularization on/off ($\lambda = 36$, the established best single-axis point) and pruning on/off ($p = 40\%$, a starting guess with no prior data point on this axis, mirroring VWW's choice).

Follow-up (lighter settings). Motivated by VWW's finding that a lighter joint configuration (λ and p each roughly halved) substantially reduced the accuracy cost, three lighter points were tested: $\lambda = 15$ and $p = 20\%$ individually and jointly.

Diagnostic (longer fine-tuning). The pilot's aggressive joint configuration ($\lambda = 36$, $p = 40\%$) showed a very large accuracy drop (Section 7); to determine whether this reflected an insufficient Phase-2 budget or a more fundamental problem, the same configuration was re-run with Phase 2 lengthened from 5 to 20 epochs.

Table 3: All configurations run. λ is the normalized Hoyer-Square strength; p is the structured prune target.

Tag	λ	p	Phase-2 epochs
`baseline`	0	0%	–
`activation_only`	36	0%	–
`structured_only`	0	40%	5
`joint`	36	40%	5
`activation_light`	15	0%	–
`structured_light`	0	20%	5
`joint_light`	15	20%	5
`joint_long_finetune`	36	40%	20

All runs used GPU 1 of a shared training server; the full experiment (8 configurations) took well under two hours end to end.

7 Results

Table 4: Full results. Accuracy drop is relative to `baseline` (99.72%). Parameter reduction is relative to `baseline`'s 1,156,054 parameters. Activation sparsity is the float-model measurement (Section 5.3) and is *not* directly comparable to the prior experiment's Akida-measured numbers.

Tag	λ	p	Accuracy	Acc. drop	Act. sparsity	Params (reduction)
`baseline`	0	0%	99.72%	–	51.7%	1,156,054 (–)
`activation_only`	36	0%	98.31%	1.42pt	83.1%	1,156,054 (0%)
`activation_light`	15	0%	98.86%	0.87pt	76.8%	1,156,054 (0%)
`structured_only`	0	40%	98.32%	1.40pt	56.9%	711,624 (38.4%)
`structured_light`	0	20%	99.50%	0.22pt	55.0%	913,771 (21.0%)
`joint`	36	40%	80.15%	19.58pt	81.4%	711,624 (38.4%)
`joint_long_finetune`	36	40%	93.48%	6.24pt	83.2%	711,624 (38.4%)
`joint_light`	15	20%	97.81%	1.92pt	75.9%	913,771 (21.0%)

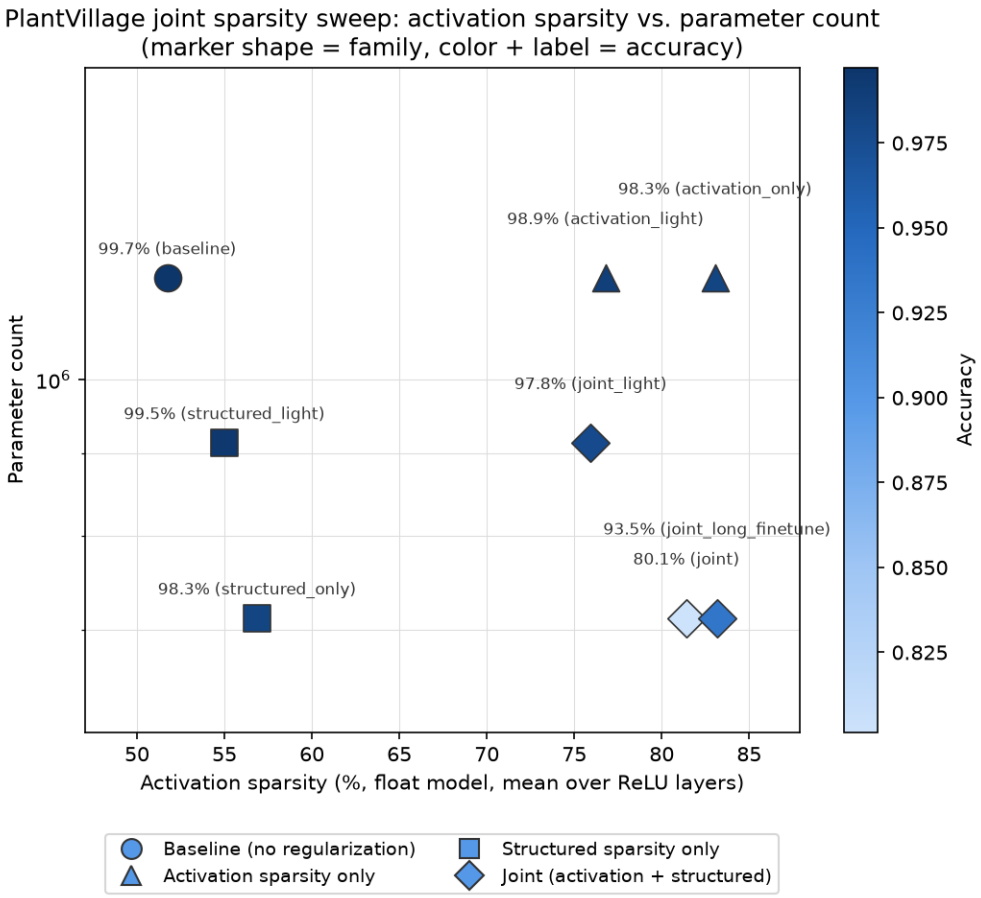

Figure 1: Activation sparsity vs. parameter count for all eight configurations. Marker shape encodes family (circle: baseline, triangle: activation sparsity only, square: structured sparsity only, diamond: joint/joint variants); marker color and the accompanying label encode accuracy. Note the two diamonds at the same parameter count (right side): the darker `joint` (80.1%, 5 fine-tune epochs) and the lighter `joint_long_finetune` (93.5%, 20 fine-tune epochs).

7.1 An aggressive joint configuration is antagonistic, not complementary

Relative to `baseline`, `activation_only` alone costs 1.42 points and `structured_only` alone costs 1.40 points; a naive additive expectation predicts ∼2.81 points for both together. The aggressive `joint` configuration instead costs **19.58 points** – an order of magnitude worse than either mechanism alone, and the opposite of what the VWW study found (there, the joint cost was *less* than the sum of its parts).

7.2 Tracing the cause: a pruning-recovery problem, not unstable joint training

Phase 1 training (both regularizers active simultaneously) was entirely well-behaved: validation accuracy climbed smoothly from 79.9% to 98.3% over 10 epochs, with no sign of instability. The collapse happens precisely at the channel-pruning step: the first epoch of Phase 2 opens at only 59.9% validation accuracy (down from 98.3% immediately beforehand, before any further training has occurred), and the default 5 fine-tuning epochs recover only to 78.9%. This indicates that channel pruning driven by BN-γ magnitude, performed while the activation regularizer is simultaneously

suppressing the network's internal activations, removes channels that matter substantially more than the same 40% removed under `structured_only`'s undisturbed γ ranking.

7.3 Most, but not all, of the damage is recoverable with more fine-tuning

`joint_long_finetune` – identical (λ, p) to the aggressive `joint`, but with Phase 2 lengthened from 5 to 20 epochs – recovers to **93.48%**, a large improvement over 80.15% that confirms the original number was substantially an under-trained-recovery artifact rather than irreversible channel damage. Even so, a 6.24-point cost remains at four times the fine-tuning budget, versus ∼1.4 points for either mechanism alone – so some genuine antagonism between the two mechanisms persists at these aggressive settings, just far less severe than the 5-epoch figure suggested.

7.4 A lighter joint configuration avoids the problem almost entirely

`joint_light` ($\lambda = 15$, $p = 20\%$), using the same modest 5-epoch fine-tuning budget that left the aggressive configuration badly undertrained, reaches **97.81%** accuracy (a 1.92-point drop) with 75.9% activation sparsity and a 21.0% parameter reduction – better on every axis than `joint_long_finetune` achieved even with four times the fine-tuning time at the aggressive settings. Whatever makes aggressive joint pruning hard to recover from is evidently much less of a problem at lighter regularization/pruning strengths.

7.5 Recommendation

`joint_light` ($\lambda = 15$, $p = 20\%$) is the clear best operating point found in this study. Unlike VWW, where every joint configuration tested was a reasonable trade and the lighter one simply used the accuracy budget more efficiently, PlantVillage's aggressive joint configuration is a genuine trap: it costs over 13 points more accuracy than `joint_light` even after quadrupling its recovery budget. The practical guidance for this model is to stay in the lighter joint regime rather than push both regularization knobs hard simultaneously.

8 Discussion and limitations

Why does the interaction differ from VWW? This report does not establish a confirmed mechanism, only a well-supported hypothesis: the BN-γ importance ranking that guides pruning may become less reliable once a strong activation regularizer is simultaneously reshaping the network's internal representations, so pruning "the lowest-γ channels" no longer reliably means "the least task-relevant channels" when that regularizer is active. Why this effect would be more severe on PlantVillage than on VWW is not established here – plausible contributing factors include PlantVillage's much larger per-layer size imbalance (Section 5.1) and its considerably higher output cardinality (38 classes vs. 2), but confirming either would require further experiments (e.g. comparing which specific channels get pruned under `structured_only` vs. `joint` at matched prune targets).

Float-vs-hardware sparsity gap and no latency validation. As with VWW, the float-model activation-sparsity measurements used throughout this report are not on the same footing as the original single-axis experiment's Akida-hardware/software-simulator-measured figures, and this study makes no claim about actual Akida 1 latency or power – the parameter-count reduction is a memory-footprint argument only, since Akida 1 does not accelerate on weight sparsity.

Unexplored surface. `joint_long_finetune` was only tested at 20 Phase-2 epochs; a longer fine-tune might close more of the remaining 6.24-point gap, or might plateau before that. The reverse combinations – light activation regularization with aggressive pruning, or aggressive activation regularization with light pruning – have not been tested, so it is not yet known whether the antagonism requires both knobs to be turned up together or whether either one alone at full strength is already sufficient to make pruning hard to recover from.

9 Conclusion

Repeating the joint activation-sparsity and structured-sparsity investigation on PlantVillage produced a materially different finding than on VWW: rather than the two mechanisms being complementary, an aggressive joint configuration here is actively antagonistic, costing an order of magnitude more accuracy than either mechanism alone. Diagnosis shows this is substantially a fine-tuning-budget problem – quadrupling the post-prune recovery time recovers most of the gap – but a genuine residual antagonism remains at aggressive settings even so. A lighter joint configuration ($\lambda = 15$ activation regularization, 20% structured pruning) sidesteps the problem almost entirely, reaching 97.81% accuracy with 75.9% activation sparsity and a 21.0% parameter reduction using the same modest fine-tuning budget that left the aggressive configuration badly undertrained. The practical recommendation — favor a lighter joint configuration — matches VWW's, but this study shows that conclusion should not be assumed to transfer for the *same reason*: on VWW it reflected a smoothly diminishing-returns curve, while on PlantVillage it reflects an outright trap at aggressive settings that a lighter configuration avoids.

References

[1] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang. Learning Efficient Convolutional Networks through Network Slimming. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

[2] H. Yang, W. Wen, and H. Li. DeepHoyer: Learning Sparser Neural Network with Differentiable Scale-Invariant Sparsity Measures. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861*, 2017.

[4] D. P. Hughes and M. Salathé. An Open Access Repository of Images on Plant Health to Enable the Development of Mobile Disease Diagnostics. *arXiv preprint arXiv:1511.08060*, 2015.